

Plataforma Moderna de Ingeniería de Datos: Arquitectura, Implementación y Análisis de DAGs con Apache Airflow

Diego Alejandro Escobar Barahona

Daniella Marissa Navarro Araniva

Level Up

Docente: Diego Menendez

Repositorio: https://github.com/bapatata688/dags_bi

Junio 2026

Resumen

El presente documento describe el diseño, la implementación y el análisis de una Plataforma Moderna de Ingeniería de Datos construida sobre Apache Airflow como motor de orquestación. La plataforma se organiza en cinco Directed Acyclic Graphs (DAGs) que representan las etapas completas de un pipeline de datos empresarial: ingesta desde sistemas ERP/CRM, ingesta desde APIs externas, transformaciones en arquitectura Lakehouse (capas Bronze, Silver y Gold), validaciones de calidad y observabilidad, y despliegue de modelos de Machine Learning junto con actualización de dashboards ejecutivos. Se analiza la estructura de carpetas del proyecto, la justificación técnica de cada módulo, los patrones de diseño aplicados (Fan-Out/Fan-In, pipeline secuencial, callbacks de error) y las conclusiones derivadas de cada DAG. El trabajo evidencia cómo una arquitectura de datos bien orquestada reduce la latencia operativa, garantiza la trazabilidad de los datos y sienta las bases para decisiones empresariales basadas en información confiable.

Palabras clave: Apache Airflow, DAG, ingeniería de datos, Lakehouse, ETL, MLOps, calidad de datos, orquestación de pipelines.

Introducción

Las organizaciones modernas generan volúmenes crecientes de datos provenientes de sistemas heterogéneos: ERP, CRM, plataformas de comercio electrónico, redes de publicidad digital y sistemas de atención al cliente. La capacidad de integrar, transformar y poner a disposición dichos datos de forma confiable y oportuna constituye una ventaja competitiva crítica. Apache Airflow (Airflow Software Foundation, 2024) se ha consolidado como el estándar de facto para la orquestación de pipelines de datos en entornos empresariales, gracias a su modelo basado en grafos acíclicos dirigidos (DAGs), su interfaz web para monitoreo y su amplio ecosistema de operadores.

El proyecto que se documenta en este informe implementa una plataforma completa de ingeniería de datos compuesta por cinco DAGs interdependientes que cubren el ciclo de vida completo del dato: desde la extracción en bruto hasta el despliegue de modelos predictivos y la actualización de indicadores de negocio. El objetivo principal es demostrar la aplicación práctica de los principios de ingeniería de datos en un contexto empresarial simulado, justificando cada decisión arquitectónica con fundamentos técnicos y de negocio.

Estructura del Proyecto

Estructura de Carpetas

El proyecto sigue una organización plana de archivos en el directorio raíz de DAGs de Apache Airflow, siguiendo la convención de nomenclatura numerada que refleja el orden de ejecución del pipeline. La estructura es la siguiente:

Archivo / Directorio	Descripcion	Capa
dag_01_ingestion_erp_crm.py	Ingesta desde SAP (ERP) hacia Data Lake	Raw
dag_02_ingestion_apis.py	Ingesta desde APIs externas (Fan-Out/Fan-In)	Raw
dag_03_lakehouse_transformations.py	Transformaciones Bronze/Silver/Gold	Silver/Gold
dag_04_calidad_observabilidad.py	Validaciones de calidad y monitoreo	Governance
dag_05_mlops_dashboards.py	MLOps y actualizacion de dashboards BI	Analytics

Tabla 1. Estructura de archivos del proyecto.

La ausencia de subdirectorios en la raíz de DAGs responde a una decisión de simplicidad operativa: Airflow descubre automáticamente todos los archivos .py en el directorio dags/, por lo que mantener un nivel plano reduce la complejidad de configuración. Para proyectos de mayor escala se recomienda agrupar los DAGs en subdirectorios por dominio de negocio (ingestion/, transformation/, quality/, mlops/), acompañados de un directorio utils/ con funciones compartidas y un directorio tests/ con pruebas unitarias de las funciones de transformacion.

Estructura del Código y Justificación Técnica

DAG 01: Ingesta ERP/CRM (dag_01_ingestion_erp_crm.py)

Este DAG implementa el patrón más sencillo: una cadena secuencial de tres tareas que modelan el flujo de extracción desde SAP ERP hacia el Data Lake.

Patrón de diseño: Pipeline secuencial

La dependencia lineal conectar >> extraer >> guardar es deliberada. En un sistema ERP como SAP, la extracción de datos requiere una conexión establecida y autenticada antes de procesar registros, y los datos deben estar completamente extraídos antes de ser escritos en el Data Lake. Un patrón paralelo sería incorrecto aquí porque las tareas son estrictamente dependientes del estado generado por la tarea anterior.

```
conectar = PythonOperator(task_id='conectar_sap', ...) extraer =  
PythonOperator(task_id='extraer_datos_erp', ...) guardar =  
PythonOperator(task_id='guardar_data_lake', ...) conectar >> extraer >> guardar
```

Figura 1. Dependencias del DAG 01.

Justificación empresarial

La extracción diaria de 10,000 registros del ERP garantiza que el Data Lake siempre contenga los datos operativos del día anterior, permitiendo a los equipos de analítica trabajar con información actualizada. Los reintentos configurados (retries=2, retry_delay=5 minutos) cubren interrupciones transitorias de red comunes en integraciones con sistemas SAP.

DAG 02: Ingesta APIs Externas (dag_02_ingestion_apis.py)

Este es el DAG más complejo del proyecto en términos de diseño. Implementa el patrón Fan-Out / Fan-In para integrar tres APIs externas de forma paralela: Shopify (E-Commerce), Facebook Ads (Marketing) y Zendesk (Soporte al Cliente).

Patrón de diseño: Fan-Out / Fan-In

Una tarea inicial de validación (validate_api_tokens) actúa como nodo de control y verifica la configuración de tokens y cuotas antes de iniciar las extracciones. Las tres tareas de extracción corren en paralelo (fan-out), aprovechando los workers independientes de Airflow para reducir el tiempo total del pipeline. La tarea de consolidación (consolidate_api_results) actúa como fan-in con trigger_rule='all_done', lo que significa que se ejecuta aunque alguna API haya fallado, permitiendo registrar un resultado parcial en lugar de perder toda la información del

proceso.

```
task_validate_tokens >> [ task_extract_shopify, task_extract_facebook,
task_extractZendesk, ] >> task_consolidate
```

Figura 2. Patron Fan-Out/Fan-In del DAG 02.

Mecanismos técnicos implementados

Mecanismo	Implementacion	Proposito
Paginacion simulada	<code>_simulate_api_call(endpoint, page, source)</code>	Cubrir volúmenes reales de API
Almacenamiento Raw	<code>_save_to_raw_layer(data, source, endpoint)</code>	Particion por fecha en Data Lake
Comunicacion entre tareas	<code>context['ti'].xcom_push/pull</code>	Metadata de auditoría entre tareas
Rate limit awareness	<code>retry_delay=10 min, retries=3</code>	Respetar quotas de APIs externas
Auth por API	OAuth2/Token/BasicAuth por sistema	Seguridad sin credenciales en código

Tabla 2. Mecanismos técnicos del DAG 02.

DAG 03: Transformaciones Lakehouse (`dag_03_lakehouse_transformations.py`)

Este DAG implementa la arquitectura Medallion (Bronze/Silver/Gold), el patron estandar de organizacion de datos en plataformas Lakehouse modernas (Databricks, 2023). Recibe los datos crudos de los DAGs 01 y 02 y los transforma en datasets analiticos de alta calidad.

Capas de la arquitectura Medallion

Capa	Path en Data Lake	Descripcion	Funcion en DAG 03
Bronze	<code>/data_lake/raw/</code>	Datos crudos tal como llegaron	Entrada del pipeline
Silver	<code>/data_lake/processed/</code>	Datos limpios, estandarizados	<code>clean_erp_crm_data,</code> <code>clean_billing_risk_data,</code> <code>clean_external_apis_data</code>
Gold	<code>/data_lake/curated/</code>	Datasets analiticos y KPIs de negocio	<code>build_sales_analytics_dataset,</code> <code>build_customer_360_dataset,</code> <code>build_marketing_roi_dataset</code>

Tabla 3. Arquitectura Medallion implementada en el DAG 03.

Justificacion empresarial

La separacion en capas permite que distintos equipos consuman el nivel de detalle que necesitan: el equipo de ingenieria accede a Bronze para depuracion, los analistas de datos trabajan con Silver para analisis ad hoc, y los ejecutivos y modelos de ML consumen Gold.

Esta arquitectura tambien facilita la rerreprocesamiento: si se detecta un error en las reglas de negocio de Gold, solo es necesario volver a ejecutar las transformaciones desde Silver, sin necesidad de re-extraer los datos de los sistemas fuente.

DAG 04: Calidad y Observabilidad (dag_04_calidad_observabilidad.py)

Este DAG implementa un pipeline secuencial de validaciones de calidad de datos seguido de una tarea de observabilidad del pipeline. La secuencialidad es intencional: cada validacion debe completarse exitosamente antes de proceder a la siguiente, garantizando que los datos que avanzan en el pipeline cumplen los criterios de calidad definidos.

Validaciones implementadas

Task ID	Funcion	Que valida
nulos	validar_nulos()	Campos criticos sin valores nulos
duplicados	validar_duplicados()	Registros unicos por clave primaria
rangos	validar_rangos()	Valores dentro de rangos de negocio
integridad	validar_integridad()	Integridad referencial entre tablas
observabilidad	observabilidad()	Metricas SLA, latencia y estado de logs

Tabla 4. Pipeline de validaciones del DAG 04.

Callback de fallo y alertas

La funcion `alerta_fallo(context)` actua como callback de error registrado en `on_failure_callback` de los `default_args`. Esto significa que Airflow la invocara automaticamente ante cualquier fallo de tarea, sin necesidad de codigo adicional en cada `PythonOperator`. En produccion, esta funcion enviaria notificaciones a canales de Slack, Teams o sistemas de alertas como PagerDuty, proporcionando el `task_id` fallido para acelerar el diagnostico.

```
def alerta_fallo(context): print('fallo detectado en:',
context['task_instance'].task_id) print('enviando notificacion a sistema de
alertas...') default_args = { 'on_failure_callback': alerta_fallo, ... }
```

Figura 3. Implementacion del callback de error en DAG 04.

DAG 05: MLOps y Dashboards (dag_05_mlops_dashboards.py)

El DAG final cierra el ciclo de vida del dato al transformar los datasets analiticos de la capa Gold en valor de negocio tangible: modelos predictivos en produccion y dashboards ejecutivos

actualizados.

Pipeline MLOps implementado

Etapa	Task ID	Accion	Metricas de referencia
Entrenamiento	train	Modelos de ventas, churn y segmentacion	Datos desde Data Warehouse
Evaluacion	evaluate	Validacion de metricas de desempenho	Accuracy: 0.87, RMSE: 12.4, Silhouette: 0.71
Despliegue	deploy	Publicacion en produccion y monitoreo de drift	Reentrenamiento automatico
Dashboards	dashboards	Actualizacion Power BI, Tableau, Looker	KPIs: ventas, churn, inventario

Tabla 5. Pipeline MLOps del DAG 05.

Implementacion en Python

Convenciones de Codigo Aplicadas

El proyecto aplica convenciones consistentes de codificacion a lo largo de los cinco DAGs, garantizando mantenibilidad y legibilidad:

Convencion	Detalle
Nomenclatura de archivos	dag_NN_nombre_descriptivo.py (numerado por orden de ejecucion)
Nomenclatura de funciones	snake_case en espanol/ingles segun contexto de dominio
Documentacion	Docstrings con Args/Returns en funciones publicas (DAG 02 y 03)
Separacion de secciones	Comentarios de seccion con separadores visuales (===)
Logging	logging.getLogger(__name__) en lugar de print() (DAG 02 y 03)
Constantes centralizadas	API_CONFIG, RAW_LAYER_PATH, SOURCE_* como modulo-nivel
Default args	dict centralizado reusado por todas las tareas del DAG
Bloque __main__	Permite ejecucion standalone para pruebas sin Airflow
Manejo de errores	try/except con logging del error y re-raise para Airflow

Tabla 6. Convenciones de codigo del proyecto.

Patron de Ejecucion Standalone

Todos los DAGs incluyen un bloque `if __name__ == '__main__':` que permite ejecutar las funciones de negocio directamente desde la linea de comandos sin necesidad de levantar un entorno Airflow completo. Este patron es especialmente util durante el desarrollo y la depuracion, ya que permite verificar la logica de cada funcion de forma aislada antes de integrarla al scheduler de Airflow.

```
if __name__ == '__main__': print('===== EJECUCION DE PRUEBA DAG ERP =====')
conectar_sap() extraer_datos_erp() guardar_data_lake() print('===== PROCESO
FINALIZADO =====')
```

Figura 4. Patron de ejecucion standalone presente en todos los DAGs.

Gestion de Dependencias entre DAGs

La plataforma esta disenada con un orden de ejecucion explicito que refleja el flujo de datos entre capas. Aunque en el codigo actual no se implementa un `TriggerDagRunOperator` (mencionado en los comentarios del DAG 02), la arquitectura prevee esta integracion para

produccion:

DAG	Horario	Depende de	Activa
DAG 01	02:00 AM	Sistemas ERP/CRM	DAG 03
DAG 02	03:00 AM	APIs externas	DAG 03
DAG 03	05:00 AM	DAG 01 y DAG 02	DAG 04
DAG 04	06:00 AM	DAG 03	DAG 05
DAG 05	07:00 AM	DAG 04	Dashboards BI

Tabla 7. Orden de ejecucion y dependencias entre DAGs.

Analisis Empresarial de la Plataforma

Valor de Negocio por DAG

DAG 01: Continuidad operativa del ERP

La extraccion diaria automatizada del ERP elimina la dependencia de procesos manuales de exportacion de datos, reduciendo el riesgo de errores humanos y garantizando que los datos operativos esten disponibles para analisis cada manana. Para una empresa salvadorena del sector retail o manufactura, esto puede significar la diferencia entre tomar decisiones de compra o produccion basadas en datos del dia anterior versus datos de la semana pasada.

DAG 02: Vision 360 del cliente y canal digital

La integracion paralela de Shopify, Facebook Ads y Zendesk proporciona una vision unificada del ciclo de vida del cliente digital: desde el primer contacto publicitario (Facebook Ads), la conversion en compra (Shopify), hasta la experiencia post-venta (Zendesk). Esta integracion es el fundamento de estrategias de marketing basadas en datos, como la atribucion de ingresos a campanas especificas o la identificacion de segmentos de clientes con alta tasa de tickets de soporte.

DAG 03: Datos confiables para decisiones estrategicas

La arquitectura Medallion garantiza que los datos que llegan a la capa Gold han pasado por un proceso riguroso de limpieza y estandarizacion. El dataset de Sales Analytics (ingresos netos, margen por producto), el Customer 360 (LTV, segmento de riesgo) y el Marketing ROI (ROAS por campana) son los insumos directos para las reuniones de direccion y las decisiones de inversion. Un error en esta capa tiene impacto directo en la estrategia empresarial, de ahi la importancia del DAG 04.

DAG 04: Gobierno de datos y confianza en la informacion

La calidad de datos es el pilar de la confianza organizacional en los sistemas de informacion. Un solo error de datos no detectado, como un campo nulo en el monto de una factura o un registro duplicado en el inventario, puede generar decisiones erroneas con consecuencias financieras significativas. El DAG 04 actua como la ultima linea de defensa antes de que los datos lleguen a los consumidores finales, incluyendo el motor de machine learning.

DAG 05: Inteligencia artificial al servicio del negocio

Los tres modelos implementados en el DAG 05 abordan los principales desafíos de negocio de una empresa moderna: la prediccion de ventas permite optimizar el inventario y la cadena de suministro, el modelo de churn identifica clientes en riesgo de abandono para acciones de retencion proactiva, y la segmentacion de clientes habilita estrategias de marketing personalizado. Los dashboards en Power BI, Tableau y Looker democratizan el acceso a estos insights, llevandolos desde el equipo de data science hasta los tomadores de decision sin conocimientos tecnicos.

Indicadores de Salud de la Plataforma

La plataforma incluye configuraciones que permiten monitorear su salud operativa de forma continua. El DAG 04 reporta las siguientes metricas de referencia para produccion:

Metrica	Valor objetivo	Significado empresarial
SLA del pipeline	99%	Disponibilidad de datos para los equipos de negocio
Latencia del pipeline	< 5 horas	Datos del dia disponibles antes de la jornada laboral
Reintentos promedio	< 0.5 por tarea	Estabilidad de las conexiones a sistemas fuente
Tasa de validacion	100% sin alertas	Calidad de datos garantizada en capa Gold
Accuracy del modelo	0.87	Confiabilidad de predicciones para decision de negocio

Tabla 8. KPIs operativos de la plataforma de datos.

Conclusiones

Conclusiones por DAG

DAG 01 — Ingesta ERP/CRM

El DAG 01 demuestra que un pipeline de ingesta desde sistemas ERP puede implementarse con un nivel mínimo de complejidad cuando las tareas son estrictamente secuenciales. La elección del patrón pipeline lineal es correcta para este caso de uso: la dependencia entre la conexión, la extracción y el almacenamiento es total e ineludible. La configuración de reintentos (retries=2, delay=5 minutos) es suficiente para cubrir interrupciones transitorias sin sobrecargar el sistema SAP. Para escalar este DAG a volúmenes superiores a los 10,000 registros diarios, se recomienda implementar extracción incremental basada en timestamps de modificación, reduciendo el volumen transferido y el impacto en el sistema fuente.

DAG 02 — Ingesta APIs Externas

El DAG 02 es el más sofisticado del proyecto y el que mejor refleja los desafíos reales de la integración con APIs externas. El patrón Fan-Out/Fan-In es la elección óptima porque maximiza el paralelismo (las tres APIs son independientes entre sí) mientras mantiene un punto único de control al inicio y al final. El uso de XCom para comunicar metadata entre tareas es una práctica correcta de Airflow, aunque para volúmenes de datos grandes se debe evitar pasar los datos mismos por XCom (límite de 48KB por defecto) y en su lugar pasar solo rutas o referencias. El trigger_rule='all_done' en la tarea de consolidación es una decisión de diseño crítica que permite obtener información parcial del pipeline incluso cuando alguna API falla, lo cual es preferible a perder todo el contexto de ejecución.

DAG 03 — Transformaciones Lakehouse

La arquitectura Medallion implementada en el DAG 03 es el estándar industrial para plataformas de datos modernas y su correcta implementación es crítica para la calidad de los datos analíticos. La separación en funciones por fuente de datos (clean_erp_crm_data, clean_billing_risk_data, clean_external_apis_data) sigue el principio de responsabilidad única y facilita el mantenimiento independiente de cada transformación. La capa Gold con datasets especializados por dominio (ventas, cliente, marketing) es la base correcta para un Data Warehouse orientado al negocio. Una mejora recomendada para producción es implementar pruebas de calidad de datos con frameworks como Great Expectations o dbt tests directamente en las transformaciones Silver, antes de promover datos a Gold.

DAG 04 — Calidad y Observabilidad

El DAG 04 cierra una brecha critica en cualquier plataforma de datos: la ausencia de validaciones formales de calidad. La implementacion de cuatro dimensiones de calidad (nulos, duplicados, rangos e integridad) cubre los casos de fallo mas comunes en pipelines de datos empresariales. El mecanismo de callback `on_failure_callback` es una característica de Airflow frecuentemente subutilizada pero de alto valor operativo, ya que convierte cada fallo de tarea en una notificación accionable. La inclusion de una tarea de observabilidad como ultimo paso del DAG es especialmente valiosa: permite registrar el estado final del pipeline de forma estructurada, creando un historial auditable del desempeño de la plataforma.

DAG 05 — MLOps y Dashboards

El DAG 05 representa la culminacion del ciclo de valor de los datos: su transformacion en inteligencia de negocio accionable. La inclusion del monitoreo de data drift en el paso de despliegue es una practica de MLOps madura que previene la degradacion silenciosa de los modelos ante cambios en los patrones de datos de entrada. Las metricas reportadas (Accuracy=0.87, RMSE=12.4, Silhouette=0.71) son valores de referencia razonables para un primer modelo en produccion, aunque deberian evaluarse en el contexto del negocio especifico. La actualizacion automatica de dashboards en tres plataformas de BI diferentes (Power BI, Tableau, Looker) sugiere que la organizacion objetivo tiene distintos equipos con preferencias de visualizacion diferentes, lo cual es comun en empresas en transicion hacia una cultura de datos.

Conclusion General

La Plataforma Moderna de Ingeniería de Datos documentada en este informe demuestra que es posible implementar un ciclo de vida completo del dato, desde la extraccion en bruto hasta el despliegue de modelos predictivos, utilizando unicamente Apache Airflow como motor de orquestacion y Python como lenguaje de implementacion. La arquitectura de cinco DAGs especializados respeta el principio de separacion de responsabilidades (separation of concerns), lo que facilita el mantenimiento, la depuracion y la evolucion independiente de cada etapa del pipeline.

Desde una perspectiva empresarial, esta plataforma representa una inversion en infraestructura de datos que habilita capacidades de analisis avanzado, inteligencia artificial y gobierno de datos que son indispensables para la competitividad en el entorno empresarial actual. La combinacion de automatizacion, calidad de datos garantizada y modelos predictivos en

produccion constituye la base tecnica de una organizacion verdaderamente orientada a los datos (data-driven organization).

Las areas de mejora identificadas para una implementacion productiva incluyen: la adicion de pruebas unitarias para las funciones de transformacion, la implementacion de extraccion incremental en los DAGs de ingesta, la integracion con un catalogo de datos (DataHub u OpenMetadata) para trazabilidad automatica, y la adicion de TriggerDagRunOperator para establecer dependencias formales entre DAGs. Estas mejoras, aunque no implementadas en la version actual por razones de alcance del proyecto, estan previstas en la arquitectura del codigo a traves de comentarios y documentacion inline.